CAGE-2: From Decision to Effect

Consequence Assurance for Autonomous AI Systems

Roopam Sure

September 2026

Abstract

Autonomous AI systems increasingly act on external systems rather than merely produce recommendations. They can modify access rights, release payments, change infrastructure, delete records, invoke enterprise services, and initiate other operations with durable business consequences. Existing identity, authorization, policy, and runtime-control mechanisms can determine whether an agent or request may proceed, but those controls do not by themselves establish what ultimately became true in the external system.

CAGE (Control Assurance Governance Evaluation) is an assurance framework for consequential actions performed by AI agents and automated systems, designed to preserve control and evidence across the transition from proposed action to externally evidenced business consequence.

This paper introduces Consequence Assurance, an architectural model for preserving assurance across the transition from an autonomous decision to an externally verified business effect. It extends the Prebind Assurance and business-consequence-boundary model introduced in CAGE-1 and subsequently implemented publicly in CAGE-lite. In that earlier work, CAGE established that an autonomous action should not become a binding business consequence merely because an agent proposes or is authorized to perform it; the action must first cross a controlled assurance boundary supported by the required identity, standing, policy, approval, and evidence conditions. CAGE-lite operationalized this principle through explicit boundary decisions, EffectProof, Warrants, and held-to-admitted replay.

CAGE-2 formalizes and extends this earlier concept as Consequence Custody: the controlled transition from an assurance Decision into external execution, together with preservation of assurance after that transition. Implementation revealed a further problem: a system may correctly permit and dispatch an action while still being unable to establish whether the intended consequence actually became effective. This motivates the primary invariant of CAGE-2:

Decision ≠ Effect

CAGE-2 introduces a stable Consequence identity and separates Evaluation Attempts from ExecutionAttempts. It formalizes the earlier CAGE consequence-boundary custody concept as Consequence Custody. It also separates execution observations from authoritative verification, represents unresolved outcomes through EFFECT_UNKNOWN, and preserves assurance history through DecisionProof, EffectProof, and Warrants. The architecture is implemented in the open-source cage-assurance Python SDK and evaluated through conformance scenarios covering refused and narrowed actions, ambiguous execution, acknowledgement without effect, replay, and later reconciliation.

The evaluation supports a narrower claim than production correctness: within the evaluated reference implementation and scenario suite, CAGE preserves the stated semantic separations and prevents the tested unsupported transitions from being promoted into stronger assurance claims. CAGE does not replace IAM, policy engines, agent-control middleware, cloud permissions, transaction systems, or observability. Its narrower objective is to answer a different question: what can an autonomous system defensibly claim about a business consequence, given the evidence currently available?

Keywords: autonomous agents, AI assurance, agentic AI, consequence assurance, execution control, verification, AI governance, idempotency, reconciliation, auditability.

1. Introduction

The emerging generation of AI agents differs from earlier generative systems in an important way: their outputs can become actions.

An agent may not stop after recommending that a user account be suspended, a payment be released, or a cloud resource be modified. With suitable tools and permissions, it may perform the operation itself. This creates a different assurance problem. For a conventional application, authorization often answers a question such as:

Is this principal permitted to invoke this operation on this resource?

For an autonomous agent, that question remains necessary but is no longer sufficient. The requested operation may have been dynamically selected by a model, composed from several intermediate steps, or generated in response to changing context. More importantly, even after the action is authorized and dispatched, another question remains unanswered: What actually happened?

A system can know that it allowed an action without knowing whether the resulting business consequence occurred. Consider a payment agent. The agent proposes a transfer, required approvals are present, and the action is permitted. The payment API request is sent, but the client loses the connection before receiving a response.

Three interpretations remain possible: the payment was never processed; it was processed successfully; or the system cannot yet determine which is true. A binary success/failure model tends to collapse the third case into one of the first two. That can be dangerous. Interpreting uncertainty as failure may cause a retry and duplicate the payment. Interpreting acknowledgement as success may create a false audit claim when the target system never committed the operation.

This problem became apparent while implementing the Prebind Assurance concepts introduced in CAGE-1. The original model concentrated on whether an action should be allowed to cross a protected consequence boundary. Once external execution was added, it became clear that permission and effect required different semantics. This paper develops that distinction into a broader architecture. The central proposition is:

Decision ≠ Effect

A Decision is a normative claim about what CAGE permits.

An Effect is an evidence-backed claim about what external verification establishes to be true in the external system. Between them lies a sequence of events that should not be collapsed:

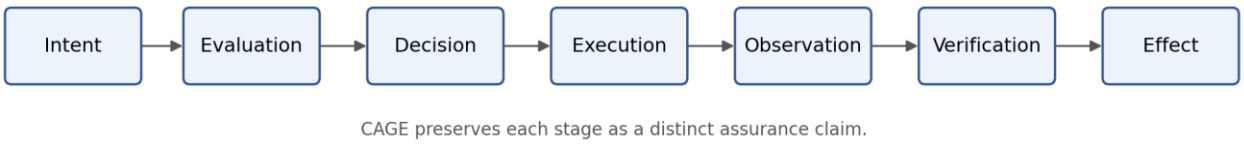


Figure 1. Decision-to-effect lifecycle. Each stage carries a different assurance meaning and should not be collapsed into a single success/failure state.

The resulting model is called Consequence Assurance.

The paper develops four connected parts of the model. It defines Consequence Assurance around the invariant Decision ≠ Effect; extends the earlier CAGE business-consequence boundary into Consequence Custody for controlled external execution; defines the semantics of execution observation, verification, Effect, uncertainty, replay, retry, and reconciliation; and preserves proof and Warrant lineage so that later evidence can strengthen the current assurance state without rewriting earlier states that were valid at the time.

CAGE-2 makes three research contributions:

1. A consequence-centric assurance model that separates permission from externally evidenced effect, while preserving a stable business Consequence across reevaluation, execution, verification, and reconciliation.
2. Consequence Custody and evidence-bounded execution semantics that keep evaluation, dispatch observation, verification, uncertainty, replay, retry, and reconciliation as distinct operations with distinct safety meanings.
3. An assurance-lineage model based on DecisionProof, EffectProof, and Warrants, together with a reference implementation and scenario-based conformance evaluation of the stated semantic invariants.

The open-source CAGE reference SDK implements these ideas. The evaluation uses conformance scenarios derived from the semantic invariants.

2. Research Questions

CAGE-2 is organized around four research questions.

RQ1 — Can permission and external effect be represented as separate assurance states?

A system should be able to state that an operation was allowed without implying that the corresponding external business state was created.

RQ2 — Can uncertain execution outcomes be preserved without converting them prematurely into success or failure?

Distributed execution frequently produces incomplete knowledge. The architecture therefore needs a first-class representation of uncertainty.

RQ3 — Can reevaluation, execution retry, and state reconciliation remain distinguishable?

These operations change different parts of the system and therefore have different safety implications.

RQ4 — Can the complete assurance history of a consequence be reconstructed without rewriting earlier states?

Later evidence should refine assurance while preserving historically correct earlier conclusions.

These questions define the scope of the work.

3. Design Principles

The architecture was developed around six principles.

3.1 Business consequence before transport operation

A network request or API call is not necessarily the correct unit of assurance. CAGE instead treats the intended business outcome as the stable object being controlled.

3.2 Permission is not evidence of execution

An ADMITTED decision means that an action may be attempted. It does not mean the effect exists.

3.3 Execution response is not external truth

A provider response describes an execution interaction. It may contribute evidence, but it should not automatically determine external effect.

3.4 Uncertainty must remain explicit

If available evidence does not establish whether the consequence occurred, the architecture should say so directly.

3.5 Re-observation must not imply re-execution

A system must be able to inspect external state again without performing the action again.

3.6 Assurance history should be append-only in meaning

Later evidence may supersede what is currently believed, but should not erase what was known previously.

Together, these principles define evidence-bounded assurance: a system should not make an assurance claim stronger than the evidence supporting it.

4. Core Terminology

CAGE-2 uses a small vocabulary to keep business intent, evaluation, execution, verification, and assurance evidence separate. The terms below are used consistently throughout the paper.

Term	Meaning in CAGE-2
Consequence	A stable representation of one intended business outcome, such as releasing a payment, granting scoped access, or deleting a resource. It is the primary unit of assurance and remains stable across reevaluation, execution attempts, verification, and reconciliation.

Term	Meaning in CAGE-2
Consequence Assurance	The overall assurance model used to determine what can defensibly be claimed about a Consequence as it moves from decision through execution and external verification.
Consequence Custody	The controlled transition from an assurance Decision into an external mutation attempt, including enforcement of the permitted effect, identity, scope, and execution conditions.
Evaluation Attempt	One evaluation of a Consequence using the evidence, approvals, standing, delegation, and context available at that time. Multiple Evaluation Attempts may refer to the same Consequence.
Decision	The normative result of evaluation: what CAGE permits to be attempted. A Decision does not establish that an external effect occurred.
ExecutionAttempt	One identified attempt to produce an external mutation corresponding to an eligible Decision. It is distinct from an Evaluation Attempt.
Effect	The evidence-backed assurance state describing what verification establishes about the external outcome. Current Effect states are BOUND, NO_BIND, and EFFECT_UNKNOWN.
DecisionProof	An assurance record describing the basis and result of a specific evaluation. It establishes what CAGE decided, not whether the external consequence occurred.
EffectProof	An assurance record tied to verification evidence and the resulting Effect state.
Warrant	A structured assurance record that brings the available Decision and Effect evidence together for a Consequence at a point in time while preserving lineage to earlier Warrants.

These terms are intentionally distinct. A Consequence is the business outcome being controlled; an Evaluation Attempt is one assessment of that outcome; an ExecutionAttempt is one attempt to produce it; and an Effect is what verification establishes about the external result. This separation supports the central CAGE-2 invariant: Decision ≠ Effect.

5. Consequence as the Unit of Assurance

A conventional software implementation often organizes activity around calls:

POST /payments

DELETE /resource/17

PATCH /users/42/roles

These identifiers are useful for execution, but they are not always the right unit for assurance. CAGE instead uses a Consequence to represent one intended business outcome. Examples include releasing payment P1042 for $25,000 to Supplier A, granting User U42 scoped operator access for 30 minutes, and deleting database object R91.

The Consequence remains stable even if the action is evaluated more than once. For example, Consequence C1 may be associated with Evaluation Attempts A1, A2, and A3 as evidence or approval changes; those reevaluations do not create a new business intention.

This produces the identity invariant:

$$\text{ConsequenceIdentity} \neq \text{EvaluationAttemptIdentity}$$

The distinction is necessary because evidence, context, approvals, or policy state may change without changing the underlying business intent. A first evaluation might result in escalation. A later evaluation might admit the same consequence after additional approval is supplied. Creating a new business identity for each reevaluation would obscure this relationship.

6. Evaluation and Decision

Each evaluation considers an Evaluation Attempt associated with a Consequence. The current CAGE implementation accepts normalized assurance inputs including Evidence, Standing, Delegation, Approval, and Context. These are interpreted by application-supplied evaluation logic; CAGE deliberately does not define a universal policy language because assurance conditions differ substantially across domains.

For example, a request for Global Administrator access for eight hours can be NARROWED to Scoped Operator access for 30 minutes; only that permitted effect may execute. These Decisions describe what CAGE allows, not external state. Therefore, ADMITTED ↛ BOUND and REFUSED ↛ NO_BIND.

7. The Decision–Effect Gap

A Decision is a normative state. An Effect is an empirical state. The former answers: What may be attempted?

The latter answers: What does the available evidence establish about what actually happened? Several shortcuts violate this distinction. Treating ADMITTED as BOUND is invalid because authorization does not imply execution. Treating an API timeout as NO_BIND is invalid because the external system may have committed before communication failed. Similarly, treating HTTP 200 as BOUND is not universally valid because an acknowledgement may indicate request acceptance rather than final business state.

CAGE therefore preserves the following semantic separations:

Semantic separation	Meaning
Decision ≠ Effect	Permission is not proof of external state.
Evaluation Attempt ≠ ExecutionAttempt	Reevaluation is not re-execution.
Adapter Result ≠ Effect	Transport observation is not authoritative state.
Execution ≠ Verification	Mutation and proof are separate stages.
Replay ≠ Execution Retry	Reevaluation does not itself mutate external state.
Reconciliation ≠ Re-execution	External state can be re-observed without dispatching again.

These distinctions collectively define the boundary between decision assurance and consequence assurance.

8. Consequence Custody

CAGE-1 [1] introduced Prebind Assurance at the business consequence boundary: consequential autonomous actions should remain under control until the required standing, evidence, policy, approval, and assurance conditions are satisfied. CAGE-lite [2] subsequently operationalized that boundary through explicit HELD, ADMITTED, and NARROWED decisions, EffectProof, Warrants, and held-to-admitted replay. CAGE-2 formalizes and extends this earlier custody concept as Consequence Custody: the controlled transition from an assurance Decision to an external mutation attempt, together with preservation of assurance after that transition.

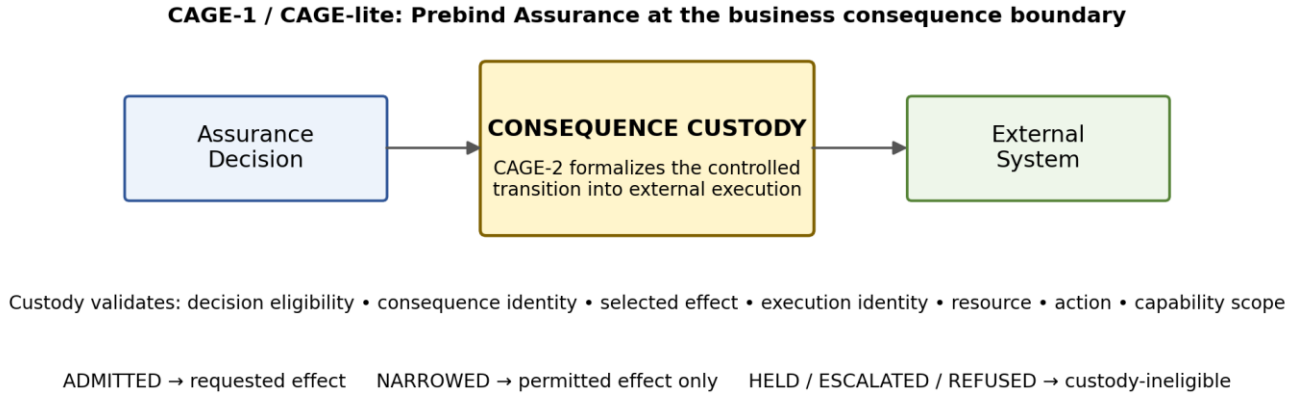


Figure 2. Consequence Custody lineage and execution boundary. CAGE-2 formalizes and extends the earlier CAGE-1/CAGE-lite consequence-boundary custody concept.

The custody boundary validates Decision eligibility, consequence identity, selected effect, execution-attempt identity, target resource, action type, and capability scope. Its execution rules are explicit:

Decision	Custody behavior
ADMITTED	Original requested effect may enter execution.
NARROWED	Only the permitted effect may enter execution.
HELD / ESCALATED / REFUSED	Custody-ineligible; no CAGE-controlled dispatch.

This prevents a NARROWED Decision from becoming advisory only. If an agent requests administrator access but is permitted only temporary scoped access, the broader request cannot be passed through custody.

9. ExecutionAttempt and Capability

Execution receives its own identity because it represents a different event from evaluation. An Evaluation Attempt is one consideration of a Consequence. An ExecutionAttempt is one identified attempt to produce an external mutation. Therefore:

$$\text{EvaluationAttemptIdentity} \neq \text{ExecutionAttemptIdentity}$$

A history can therefore contain Consequence C1 with Evaluation A1 resulting in ESCALATED, Evaluation A2 resulting in ADMITTED, and Execution E1 dispatched. This preserves the distinction between reevaluation and re-execution.

CAGE does not need to contain the underlying OAuth token, API key, password, cloud role, or secret. Those remain the responsibility of existing identity and credential systems. The assurance requirement is narrower: the execution authority presented at custody must correspond to the consequence that was evaluated.

10. Execution Observation and Verification

External mutation occurs through an EffectAdapter, which produces one of four execution observations: ACKNOWLEDGED, REJECTED, ERROR, or UNKNOWN. These describe the execution interface, not external Effect. Therefore ACKNOWLEDGED ≠ BOUND, REJECTED ≠ NO_BIND, and ERROR ≠ NO_BIND.

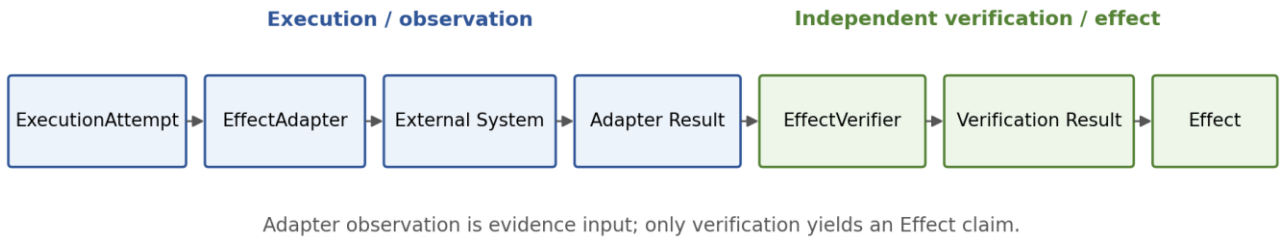


Figure 3. Execution observation and independent verification. Adapter results do not directly create Effect claims.

An acknowledgement may mean only that a request was accepted. An error may occur after a remote commit. A timeout may occur before or after processing. A rejection may relate to one execution attempt while another attempt has already changed the target state. CAGE therefore uses an independent verification stage. Here, independent means logically separate from the execution adapter; it does not require a separate organization or trust domain.

The verifier observes the target using a mechanism appropriate to the domain. Depending on the system, it may read authoritative transaction state, check an identity-provider assignment, query infrastructure state, confirm whether an object exists, or retrieve an authoritative transaction record.

CAGE does not require one universal verifier. It requires execution observation and verification to remain separate.

11. Effect States and Evidence-Bounded Assurance

CAGE currently defines three Effect states.

Effect state	Meaning
BOUND	Verification establishes that the selected effect became effective.
NO_BIND	Verification establishes that the selected effect did not become effective.
EFFECT_UNKNOWN	Available evidence is insufficient to establish either condition.

This third state is essential. If a request is dispatched, the remote system processes it, the response is lost, and the caller receives a timeout, a conventional application may record failure. CAGE instead preserves uncertainty until suitable verification resolves it.

The relevant constraints are: Lack of confirmation ≠ evidence of non-effect; request acknowledgement ≠ evidence of effect; ADMITTED ≠ BOUND; REFUSED ≠ NO_BIND; adapter success ≠ BOUND; and adapter failure ≠ NO_BIND.

This is the practical meaning of evidence-bounded assurance.

12. Verification Semantics

The verifier produces VERIFIED_BOUND, VERIFIED_NO_BIND, or INCONCLUSIVE, which map respectively to BOUND, NO_BIND, and EFFECT_UNKNOWN. Only verification can produce an Effect claim**.** Verification may consume authoritative provider state, independent readback, or other domain-appropriate evidence, but it must remain semantically distinct from the execution callback that initiated the mutation.

This does not mean the verifier is infallible. A poorly designed verifier can still produce incorrect conclusions. CAGE cannot independently guarantee the truthfulness of every external system or evidence source.

What the architecture guarantees at the semantic level is that an execution callback is not silently upgraded into a stronger external-state claim.

13. Replay, Retry, and Reconciliation

Implementation showed that three similar operations need to remain distinct.

Operation	What changes?	External mutation?
Replay	Evaluation state for the same Consequence	No
Execution retry	Creates another execution attempt	Yes
Reconciliation	Knowledge of existing external state	No

A retry carries materially greater risk because it may duplicate a consequence. For this reason, retry cannot be treated as another Evaluation Attempt.

13.1 Replay

Replay means evaluating the same Consequence again without performing a new external mutation. For example, a Consequence may first be evaluated as ESCALATED because required approval is missing. After additional approval is supplied, the same Consequence may be evaluated again and become ADMITTED.

The business intent remains unchanged; only the evaluation context or evidence changes. Replay therefore creates a new Evaluation Attempt while preserving the same Consequence identity.

Replay ≠ Execution Retry

Reevaluation does not itself mutate external state.

13.2 Execution Retry

An execution retry creates another attempt to affect external state. If an earlier ExecutionAttempt produced an uncertain outcome, a second ExecutionAttempt may duplicate the consequence if the first operation actually succeeded but the response was lost.

For this reason, execution retry carries materially greater risk than replay. It must not be treated as another Evaluation Attempt, and uncertainty should not automatically trigger re-execution.

This distinction is especially important for payments, access grants, provisioning, notifications, deletion, and other consequences that may be costly or unsafe to repeat.

13.3 Reconciliation

Reconciliation re-observes external reality without executing again. An initial AdapterExecutionResult = UNKNOWN combined with Verification = INCONCLUSIVE yields: Effect = EFFECT_UNKNOWN

If a later verifier query returns VERIFIED_BOUND, the Effect can become: BOUND without a second adapter invocation.

Reconciliation therefore changes the system's knowledge of external state, not the external state itself. This preserves the distinction:

Reconciliation ≠ Re-execution

14. Consequence-Level Idempotency

Idempotency is usually discussed as an API or transport concern. CAGE treats it at the business-consequence level. Conceptually, the same business intent should retain the same Consequence identity across reevaluation, execution, verification, and reconciliation.

Consequence identity must also remain tied to the business intent. Two operations should not collapse into one Consequence merely because a caller reused a key while changing the requested business intent. Current consequence equivalence therefore considers properties including action type, principal, resource, and requested effect.

An evaluation-attempt number, execution-retry number, agent instance, or newly proposed internal identifier should not by itself create a new business Consequence.

This model complements rather than replaces provider-side idempotency. A payment platform may still need its own durable idempotency key. CAGE's role is to preserve the logical identity of the business consequence across evaluation, execution, and verification.

15. Warrants and Assurance Lineage

Logs answer: What events were recorded?

CAGE Warrants are intended to answer a stronger question: What assurance claim can currently be supported about this Consequence? Two proof classes are maintained.

15.1 DecisionProof

A DecisionProof records the basis and result of an evaluation. It can establish what CAGE decided for a specific Evaluation Attempt. It does not prove external effect.

15.2 EffectProof

An EffectProof is tied to verification. It establishes the evidence lineage associated with an Effect state.

15.3 Warrant

A Warrant brings the available Decision and Effect evidence together into one assurance record. A Warrant containing DecisionProof with no EffectProof means that a Decision exists without a verified Effect. That is different from a Warrant containing DecisionProof plus EffectProof(EFFECT_UNKNOWN), which means execution-related evidence exists but external state remains unresolved. This distinction prevents absence of evidence from being interpreted as evidence of absence.

16. Historical Assurance Integrity

Assurance may become stronger over time. For example, Warrant W1 may record Effect = EFFECT_UNKNOWN; several minutes later, reconciliation may establish BOUND and produce Warrant W2 with previous_warrant_id = W1. W2 does not rewrite W1: the earlier Warrant remains historically correct because, at that time, the available evidence did not establish the result.

This distinction matters in incident investigation, regulated workflows, automated audit, and accountability systems because it records when a fact became supportable. A later Warrant changes what the system can currently claim; it does not change what the system could legitimately claim earlier.

17. Reference Architecture

The complete CAGE-2 lifecycle is summarized in Figure 4.

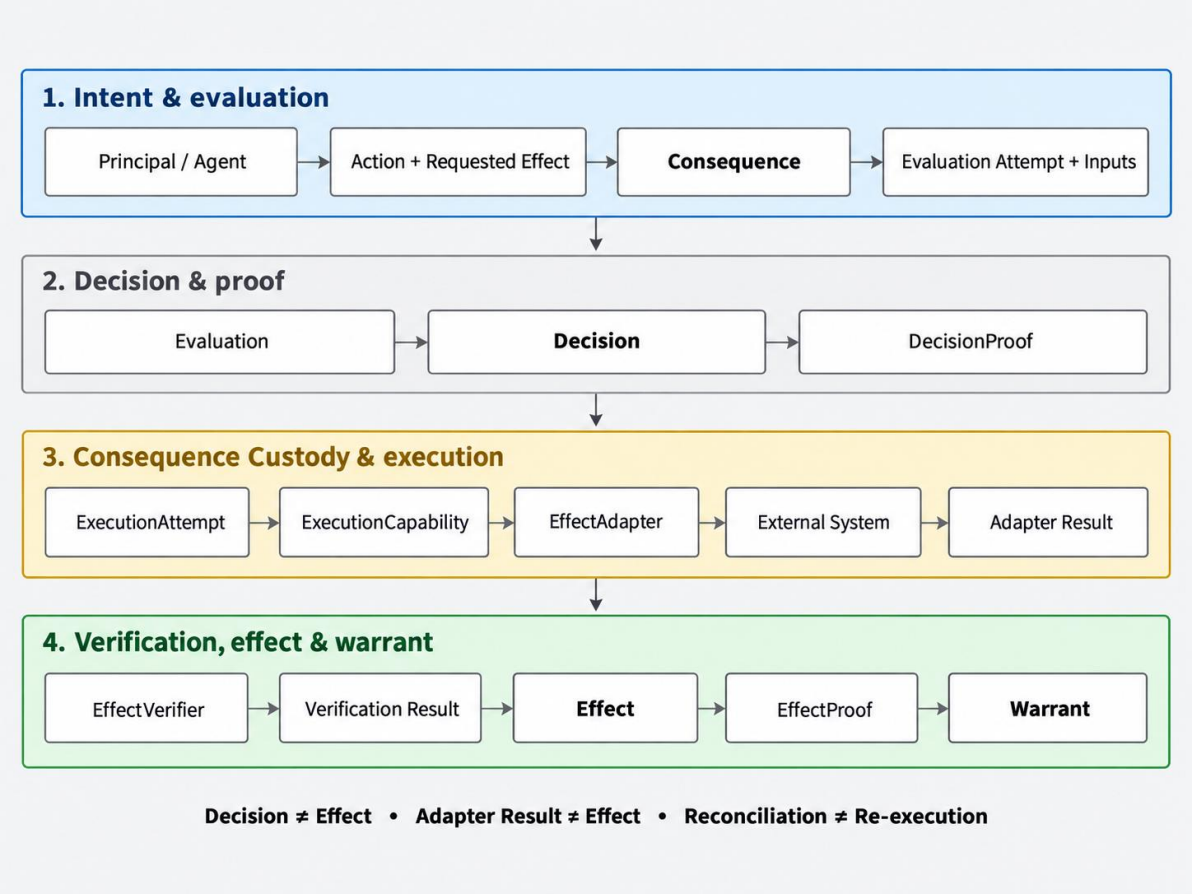


Figure 4. CAGE-2 reference architecture. The lifecycle is shown as four assurance layers, with the Consequence Custody boundary visually separated from verification and Effect.

The architecture remains semantically explicit even when its API is concise because each state makes a different assurance claim.

18. Semantic Invariants

The current implementation is designed around the following invariants.

Invariant	Semantic rule	**Interpretation**
I1 - Decision-Effect separation	Decision ≠ Effect	No Decision state directly creates an Effect claim.
I2 - No implicit execution	evaluate() does not invoke execute()	Evaluation cannot itself mutate the external system.
I3 - Attempt separation	EvaluationAttempt ≠ ExecutionAttempt	Reevaluation and external execution have distinct identities.
I4 - Verification requirement	Effect requires EffectVerificationResult	Effect claims require verification.
I5 - Evidence-bounded uncertainty	INCONCLUSIVE -> EFFECT_UNKNOWN	Uncertain evidence cannot be promoted into BOUND or NO_BIND.
I6 - Narrowed execution	NARROWED -> permitted_effect only	Only the explicitly permitted effect may enter custody.
I7 - Reconciliation non-mutating	reconcile() does not invoke EffectAdapter.execute()	Reconciliation observes existing reality.
I8 - Append-only assurance lineage	later Warrant references, not rewrites, earlier state	Historical assurance records remain intact.

These invariants are more important than any specific implementation language or provider integration.

19. Reference Implementation

The architecture is implemented in the open-source cage-assurance Python package. The developer-facing lifecycle is intentionally compact: CAGE.evaluate(...), CAGE.execute(...), CAGE.verify(...), and CAGE.reconcile(...). The public API is small, but the implementation keeps the major lifecycle concepts separate. It has distinct objects for Consequence, evaluation, Decision, custody, execution, verification, Effect, proof, and Warrant lineage.

The current implementation remains provider-neutral. The generic core deliberately does not depend directly on AWS, Azure, GCP, OpenAI, Anthropic, a particular agent runtime, a particular IAM product, or a specific policy engine. Provider-specific mechanisms should integrate at the boundaries rather than reshape the core model.

20. Evaluation Methodology

The evaluation tests whether the reference implementation follows the CAGE-2 semantic rules under representative consequence-lifecycle conditions. It is a conformance assessment, not a performance benchmark.

The evaluated reference implementation is the open-source cage-assurance Python package, release 0.7.1 [7]. The release is installable from PyPI and corresponds to the public v0.7.1 release line. For reproducibility, the source evaluated by this paper is fixed to Git tag v0.7.1 and release commit 64df7e32235129d23b3b9997aa41b4afeb90959a. The package identifier is cage-assurance==0.7.1. The verified local reference run used Windows with Python 3.13.2 and pytest; the complete test suite reported 493 passing tests. Continuous integration also exercises the package across Python 3.11, 3.12, 3.13, and 3.14. These aggregate results establish implementation health, but they are not treated as 493 independent proofs of the CAGE-2 invariants.

Unless otherwise stated, the implementation claims and scenario results in this paper refer to that tagged release and commit. The release record also states that v0.7.1 is a packaging and distribution release and does not change the runtime semantics introduced in v0.7.0.

Scenario evaluation uses deterministic local fixtures and application-supplied adapters/verifiers so that decision, dispatch observation, verification, and effect can be exercised independently without relying on a production cloud or payment provider. For each scenario, an expected semantic property is defined before execution and compared with the observed CAGE state transition. The relevant observables include Decision state, adapter invocation count, adapter observation, verification result, Effect state, identity continuity, and whether reconciliation causes another adapter invocation.

The six scenario classes are refused action without execution, narrowed execution, ambiguous external execution, acknowledgement without effect, reconciliation after uncertainty, and evaluation replay with consequence-identity continuity.

A repository traceability audit would be required to claim exact per-invariant test coverage. This paper therefore reports verified aggregate suite status and scenario-level conformance. It does not treat each of the 493 tests as an independent test of a semantic invariant.

21. Experimental Scenarios

21.1 Experiment 1 — Refused action must not execute

Input: A consequential operation lacks required assurance.

Expected:
Decision = REFUSED
Adapter invocation count = 0
Effect claim = absent

Property tested: REFUSED prevents custody execution but does not create a NO_BIND claim.

21.2 Experiment 2 — Narrowing must survive custody

Requested: Global Administrator access for 8 hours

Permitted: Scoped Operator access for 30 minutes

Expected: The adapter receives only the permitted scoped 30-minute effect.

Property tested: A NARROWED Decision is enforced at the custody boundary. The broader requested effect must not be passed to execution.

21.3 Experiment 3 — Timeout must preserve uncertainty

Input: The request is dispatched, but no reliable execution response is received.

Expected:
Adapter result = UNKNOWN
Verification = INCONCLUSIVE
Effect = EFFECT_UNKNOWN

Property tested: A communication failure must not be converted into an unsupported NO_BIND claim.

21.4 Experiment 4 — Acknowledgement must not imply effect

Input: The adapter returns ACKNOWLEDGED, but independent verification finds that the intended business state was not created.

Expected:
Adapter result = ACKNOWLEDGED
Verification = VERIFIED_NO_BIND
Effect = NO_BIND

Property tested: An execution acknowledgement does not establish external Effect. Verification remains authoritative for the Effect claim.

21.5 Experiment 5 — Reconciliation must not execute twice

Initial state:
Adapter result = UNKNOWN
Verification = INCONCLUSIVE
Effect = EFFECT_UNKNOWN

Later state:
Reconciliation returns VERIFIED_BOUND.

Expected:
Effect = BOUND
Adapter invocation count = 1
Verification count > 1

Property tested: Reconciliation may update the system's knowledge of external state without causing another execution attempt.

21.6 Experiment 6 — Replay must preserve Consequence identity

Initial state: The Consequence is evaluated and results in ESCALATED.

Later state: The same Consequence is reevaluated after additional approval is provided.

Expected:
Consequence ID = unchanged
Evaluation Attempt ID = changed

Property tested: Reevaluation creates a new Evaluation Attempt without creating a new business Consequence.

The following matrix records the primary relationship between each scenario and the semantic invariants in Section 18. It is a scenario-level mapping, not a claim of exhaustive per-invariant test traceability.

Experiment	Primary invariants	Relationship tested
1. Refused action	I1, I2	A refused Decision does not create an Effect claim and evaluation does not implicitly dispatch the action.
2. Narrowed execution	I6	Only the permitted narrowed effect may enter custody and reach the adapter.

Experiment	Primary invariants	Relationship tested
3. Timeout / ambiguous execution	I4, I5	Effect depends on verification, and inconclusive evidence remains EFFECT_UNKNOWN.
4. Acknowledgement without effect	I1, I4	An adapter acknowledgement is not promoted directly into BOUND; verification remains authoritative for the Effect claim.
5. Reconciliation after uncertainty	I5, I7	A later observation may resolve uncertainty without invoking the execution adapter again.
6. Replay / identity continuity	I2, I3	Reevaluation creates a new Evaluation Attempt while preserving the same Consequence and without implicit execution.

22. Results and Interpretation

Across the evaluated reference scenarios, the implementation preserves the distinction between permission, execution observation, verification, and effect. The important outcome is not that uncertainty is eliminated; it is that uncertainty is not silently converted into a stronger claim than the evidence supports.

Scenario	Expected semantic property	Observed implementation behavior	Result
Refused action	No custody execution; no Effect claim	Adapter is not invoked when custody is ineligible; REFUSED is not converted into NO_BIND	Conforms
Narrowed execution	Only permitted effect reaches execution	The custody path uses the permitted narrowed effect rather than the original broader request	Conforms
Ambiguous execution	UNKNOWN + INCONCLUSIVE -> EFFECT_UNKNOWN	The model preserves uncertainty rather than inferring failure	Conforms
Acknowledgement without effect	ACKNOWLEDGED must not imply BOUND	Independent verification may establish VERIFIED_NO_BIND / NO_BIND despite acknowledgement	Conforms
Reconciliation	Re-observe without second dispatch	Reconciliation can move EFFECT_UNKNOWN to a later verified state without another adapter execution	Conforms
Replay / identity continuity	Same Consequence; new evaluation Attempt	Reevaluation preserves business-consequence identity while distinguishing evaluation attempts	Conforms

One valid intermediate state is:

$$\text{Decision} = \text{ADMITTED}$$

$$\text{Execution} = \text{attempted}$$

$$\text{Adapter} = \text{UNKNOWN}$$

$$\text{Verification} = \text{INCONCLUSIVE}$$

$$\text{Effect} = \text{EFFECT_UNKNOWN}$$

This state is not an implementation failure. At that point, EFFECT_UNKNOWN is the strongest claim the available evidence supports. A later reconciliation may produce VERIFIED_BOUND and therefore BOUND without repeating execution. The reference implementation demonstrates semantic conformance for the tested scenarios; it does not, by itself, establish production-scale correctness, distributed safety across processes, or the trustworthiness of every domain-specific verifier.

22.1 Implementation Evidence Summary

Item	Evidence
Reference package	cage-assurance 0.7.1
Public distribution	PyPI
Verified local environment	Windows; Python 3.13.2
Test framework	pytest
Verified aggregate suite result	493 passed
Supported CI Python versions	3.11, 3.12, 3.13, 3.14
Evaluation type	Semantic/conformance evaluation using local fixtures and automated tests
Claim boundary	No claim of distributed durability, provider correctness, cryptographic authenticity, or per-invariant test count until separately audited

23. Comparison with Simpler Execution Models

A conventional autonomous-action lifecycle often compresses authorization, execution, and outcome into a simple success/failure path. Figure 5 contrasts that model with CAGE.

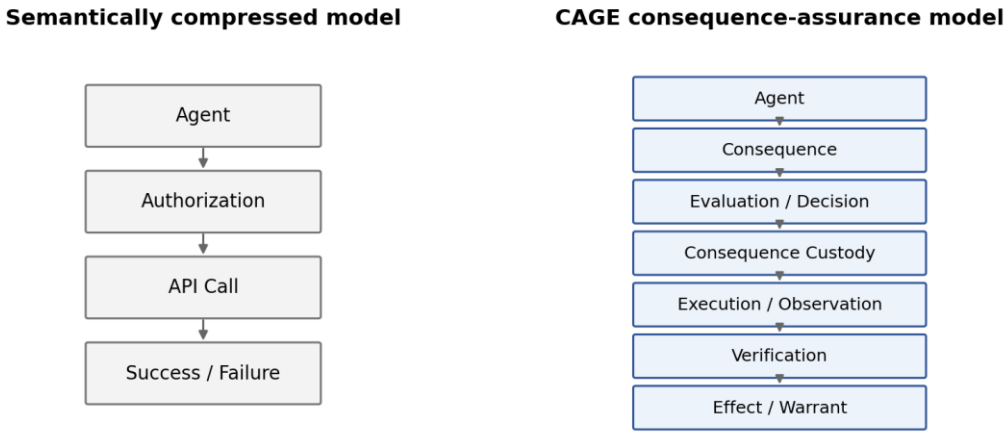


Figure 5. Comparison of a compressed authorization-to-success model with the explicit CAGE consequence-assurance lifecycle.

The advantage is not that every application requires more states. The advantage is that applications performing high-consequence operations can avoid making unsupported claims when execution and external reality diverge.

24. Related Work and Positioning

CAGE is intended to complement, not replace, identity systems, authorization engines, runtime agent-control middleware, transaction mechanisms, observability platforms, or broader AI risk-management guidance. Frameworks such as the NIST AI RMF [3], the NIST Generative AI Profile [4], and the OWASP Top 10 for Agentic Applications [6] address broader risk, governance, and security concerns. CAGE-2 focuses more narrowly on the interval from a decision to permit a consequential action through external verification of what became effective.

24.1 Agent Identity and Authorization

NIST's 2026 work on software and AI-agent identity and authorization emphasizes the need to apply identification and authorization controls to agents that access tools, applications, and data [8]. NIST's AI Agent Standards Initiative

similarly identifies secure, interoperable agent adoption as an active standards problem [5]. These efforts address an essential prerequisite: establishing who or what an agent is and what authority it may exercise. CAGE begins from that authority boundary but asks a different downstream question. A valid identity or authorization decision does not establish that the intended external effect occurred.

24.2 Runtime Agent Governance

The OWASP Agent Control Standard (ACS), published in September 2026, defines an open foundation for making agent platforms inspectable, traceable, instrumentable, and controllable at runtime through standardized middleware hooks [9]. This is an important development because it moves agent governance beyond prompt-level guidance toward runtime enforcement. CAGE is complementary to such control layers. Its distinguishing concern is not only whether a proposed tool action may cross an execution boundary, but also how the system represents the subsequent execution observation, independent verification, resulting Effect, uncertainty, and assurance lineage.

24.3 Policy Decision and Enforcement Systems

Open Policy Agent (OPA) is a general-purpose policy engine that separates policy decisions from enforcement and returns decisions to applications or enforcement points [11]. Cedar similarly evaluates whether a principal may perform an action on a resource in a given context, producing an authorization decision such as Allow or Deny [12]. The OpenID AuthZEN Authorization API 1.0 standardizes communication between Policy Decision Points and Policy Enforcement Points [10]. These systems answer whether an action is authorized. CAGE-2 addresses what happens after that decision: execution, verification, Effect, and Warrant lineage.

24.4 Distributed Execution, Idempotency, and Reconciliation

CAGE does not claim to invent idempotency, uncertain execution, retries, reconciliation, or transaction recovery. These are established distributed-systems concerns. Gray's transaction model framed atomicity, durability, and consistency as core properties for reliable state transformation [13]. Garcia-Molina and Salem's Sagas showed how long-lived work can be structured as a sequence of transactions with compensating actions rather than one global transaction [14]. Helland later described the practical importance of idempotence when messages may be retried [15] and argued that large scalable systems often cannot rely on distributed transactions spanning independently managed services [16].

CAGE-2 builds on that foundation but uses it for a different assurance question. One stable business Consequence can be reevaluated, executed, observed, verified, and later reconciled without collapsing those events into one success/failure state. Consequence-level idempotency therefore complements provider-side transaction and deduplication mechanisms rather than replacing them.

24.5 Observability, Provenance, and Audit

Observability and SIEM systems capture telemetry, logs, traces, and correlated events. Provenance research similarly asks where data came from and how it was derived; Buneman, Khanna, and Tan provide a classical treatment of why- and where-provenance [17]. Such records may become evidence for CAGE, but evidence collection and assurance interpretation remain different tasks. A log, trace, or provenance record is not automatically an assurance conclusion.

CAGE Warrants are intended to represent the strongest supported claim about a Consequence at a point in time, including cases where a Decision exists without an EffectProof or where the only defensible Effect is EFFECT_UNKNOWN. In this sense, CAGE acts as an assurance layer over evidence rather than as a replacement for telemetry or provenance infrastructure.

24.6 CAGE Positioning

System category	Primary question	Typical boundary	Relationship to CAGE
IAM / agent identity	Who or what is the actor, and what credentials/authority does it hold?	Identity, claims, credentials, authorization context	Supplies identity and authority evidence to CAGE

System category	Primary question	Typical boundary	Relationship to CAGE
Policy engine / PDP	Is this requested action permitted under policy?	Policy decision (allow/deny or richer obligations)	May supply evaluation logic or evidence; does not itself establish external Effect
Runtime agent control	May this proposed agent/tool action cross a governed execution boundary?	Runtime interception and enforcement	Complementary to Consequence Custody
Transaction / provider idempotency	How is a provider operation committed, retried, or deduplicated?	Provider-specific transaction semantics	Complements consequence-level identity and recovery
Observability / SIEM	What telemetry or events were recorded?	Logs, traces, metrics, correlated events	Can supply evidence; does not automatically produce an Effect claim
CAGE	What was permitted, what was attempted, and what does evidence establish became effective?	Decision -> Custody -> Execution -> Verification -> Effect -> Warrant	Assurance layer across the decision-to-effect interval

The distinction is intentionally narrow. CAGE is not defined by the claim that agents need runtime controls; that problem is already recognized across the ecosystem. CAGE-2 adds a separate assurance requirement: permission, dispatch observation, verification, and externally supported Effect must remain distinct, especially when execution outcomes are ambiguous.

25. Limitations

Several limitations define the current boundary of CAGE-2.

Verifier quality is one boundary. CAGE can require verification and keep it separate from execution observation, but it cannot guarantee that an external verifier or evidence source is truthful or complete.

Distributed execution is another boundary. Production deployments need durable idempotency, transaction correlation, concurrency control, and dispatch protection across processes and restarts.

The current implementation also does not provide a complete set of provider-specific production adapters.

Warrants currently provide structured assurance lineage, but they are not yet independently verifiable cryptographic attestations.

The present model focuses mainly on individual Consequences. More complex multi-agent systems may require consequence graphs, causal relationships, compensation, delegation, and dependent or conflicting sub-consequences.

CAGE also cannot prove that application-supplied evaluation logic is correct. It preserves the meaning and lineage of the resulting Decision, not the correctness of every domain rule used to produce it.

Finally, assurance metadata may contain sensitive operational or identity information and therefore requires suitable access control, retention, and minimization.

These limits separate the semantic reference model evaluated here from a production-grade assurance system.

26. Future Work

Future work has four main priorities. Distributed Consequence Custody is needed to preserve consequence-level safety across process restarts, concurrent workers, and distributed deployments through durable idempotency, execution reservation, and persistent Warrant history. Cryptographic Warrants could add canonical serialization, digital signatures, signer identity, and independent verification. Provider and agent-framework integrations are needed to test Consequence Assurance against production APIs, identity systems, cloud platforms, databases, payment systems, and agent tool protocols. Multi-agent consequence graphs may also be needed to represent parent, child, dependent, compensating, and conflicting consequences across complex autonomous workflows.

27. Discussion

CAGE-2 applies distributed-systems reasoning to the assurance problem created by autonomous actions. Distributed systems already use concepts such as idempotency, retries, reconciliation, and uncertain execution; CAGE-2 does not claim otherwise.

The difference is that autonomous systems can dynamically decide what actions to attempt, on which resources, under which context, and in what sequence. That increases the importance of preserving the relationship between machine-generated intent and externally observed outcome.

The key contribution of CAGE-2 is therefore not a new transaction protocol. It is a semantic assurance model that connects intent, permission, execution, observation, verification, effect, and proof without treating any two of them as interchangeable.

This is particularly important because autonomous systems are likely to interact with heterogeneous infrastructure.

One consequence may involve a cloud API. Another may involve an identity provider. Another may involve an enterprise database. Another may involve a payment service. CAGE does not require these systems to expose identical transaction semantics. Instead, it requires the assurance layer to remain honest about what can and cannot be established.

That requirement can be stated compactly: Do not claim more than the evidence proves.

28. Conclusion

CAGE-1 introduced Prebind Assurance: the idea that consequential autonomous actions should be evaluated before becoming operationally effective. Building that model into an executable system exposed a second problem.

Permission to act is not evidence that the intended consequence occurred. An execution response is not always proof of external state. A timeout is not necessarily failure. A retry is not the same operation as reevaluation.

A later observation should not erase earlier uncertainty. CAGE-2 addresses these problems through Consequence Assurance. Building on the consequence-boundary custody concept introduced in CAGE-1 and operationalized in CAGE-lite, CAGE-2 formalizes Consequence Custody and extends assurance through external execution, independent verification, evidence-bounded Effect states, reconciliation without re-execution, and Warrant-based assurance lineage. The architecture also introduces stable Consequence identity, explicit Decision semantics, separate ExecutionAttempts, and scoped execution capability.

The main result is architectural rather than performance-based. A trustworthy autonomous system should distinguish what it intended to do, what it was permitted to do, what it attempted, what the target system reported, and what the evidence establishes happened. These are not the same statement.

For consequential autonomous systems, treating them as the same statement creates avoidable risk. The defining invariant of CAGE-2 is therefore:

Decision ≠ Effect

A system may know what it permitted. It may know what it attempted. Only evidence can establish what became true.

References

[1] R. W. Sure, “CAGE-1: Control, Assurance, and Governance Evaluation for Enterprise Agentic AI,” arXiv preprint arXiv:2607.03510, July 2026. doi: 10.48550/arXiv.2607.03510. Available: https://arxiv.org/abs/2607.03510.

[2] R. W. Sure, “CAGE-lite v0.1.2 Product Preview: Open-Source Reference Implementation of CAGE Prebind Assurance at the Business Consequence Boundary,” GitHub release, July 21, 2026. Available: https://github.com/roopamwsure/cage-lite/releases/tag/v0.1.2.

[3] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, National Institute of Standards and Technology, 2023. doi: 10.6028/NIST.AI.100-1.

[4] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1, National Institute of Standards and Technology, 2024. doi: 10.6028/NIST.AI.600-1.

[5] National Institute of Standards and Technology, "Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation," Feb. 17, 2026. Available: https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure.

[6] OWASP GenAI Security Project, OWASP Top 10 for Agentic Applications for 2026, Dec. 9, 2025. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/.

[7] R. W. Sure, "CAGE v0.7.1 — PyPI Distribution," GitHub release, Sept. 26, 2026. Release commit: 64df7e32235129d23b3b9997aa41b4afeb90959a. Available: https://github.com/roopamwsure/cage/releases/tag/v0.7.1.

[8] H. Booth, W. Fisher, R. Galluzzo, and J. Roberts, "Accelerating the Adoption of Software and Artificial Intelligence Agent Identity and Authorization," NIST NCCoE Concept Paper, Initial Public Draft, National Institute of Standards and Technology, Feb. 5, 2026. Available: https://csrc.nist.gov/pubs/other/2026/02/05/accelerating-the-adoption-of-software-and-ai-agent/ipd.

[9] OWASP GenAI Security Project, "Agent Control Standard (ACS)," Sept. 1, 2026. Available: https://genai.owasp.org/resource/agent-control-standard-acs/.

[10] O. Gazitt, D. Brossard, and A. Tulshibagwale, eds., Authorization API 1.0, OpenID AuthZEN Working Group, OpenID Final Specification, Jan. 2026. Available: https://openid.net/specs/authorization-api-1_0.html.

[11] Open Policy Agent, "Open Policy Agent Documentation," 2026. Available: https://www.openpolicyagent.org/docs.

[12] Cedar Policy Language, "Cedar Policy Language Reference Guide," 2026. Available: https://docs.cedarpolicy.com/.

[13] J. Gray, "The Transaction Concept: Virtues and Limitations," in Proc. 7th Int. Conf. Very Large Data Bases (VLDB), 1981, pp. 144–154.

[14] H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD Int. Conf. Management of Data, 1987, pp. 249–259. doi: 10.1145/38713.38742.

[15] P. Helland, "Idempotence Is Not a Medical Condition," Communications of the ACM, vol. 55, no. 5, pp. 56–65, 2012. doi: 10.1145/2160718.2160734.

[16] P. Helland, "Life beyond Distributed Transactions: an Apostate's Opinion," in Proc. 3rd Biennial Conf. Innovative Data Systems Research (CIDR), 2007, pp. 132–141.

[17] P. Buneman, S. Khanna, and W.-C. Tan, "Why and Where: A Characterization of Data Provenance," in Database Theory — ICDT 2001, Lecture Notes in Computer Science, vol. 1973, pp. 316–330, 2001. doi: 10.1007/3-540-44503-X_20.